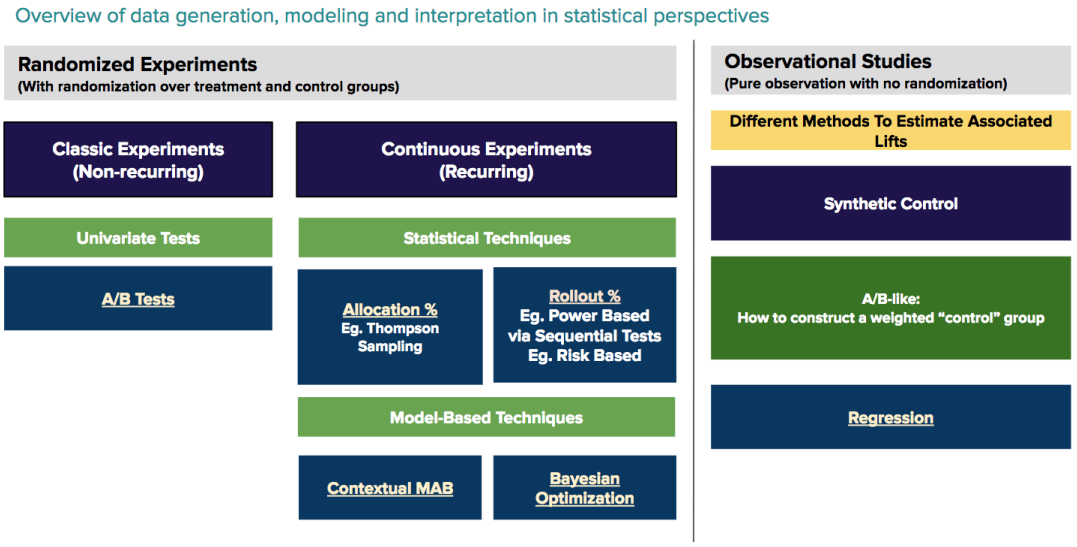
Under the Hood of Uber's Experimentation Platform

uber.com/en-PL/blog/xp

August 28, 2018



Experimentation is at the core of how Uber improves the customer experience. Uber applies several experimental methodologies to use cases as diverse as testing out a new feature to enhancing our app design.

Uber's Experimentation Platform (XP) plays an important role in this process, enabling us to launch, debug, measure, and monitor the effects of new ideas, product features, marketing campaigns, promotions, and even machine learning models. The platform supports experiments across our driver, rider, Uber Eats, and Uber Freight apps and is widely used to run A/B/N, causal inference, and multi-armed bandit (MAB)-based continuous experiments.

There are over 1,000 experiments running on our platform at any given time. For example, before Uber launched our new driver app, completely redesigned with our driver-partners in mind, it went through extensive hypothesis testings through a series of experiments conducted with our XP.

At a high level, Uber's XP allows engineers and data scientists to monitor treatment effects to ensure they do not cause regressions of any key metrics. The platform also lets users configure the universal holdout, used to measure the long-term effects of all experiments for a specific domain.

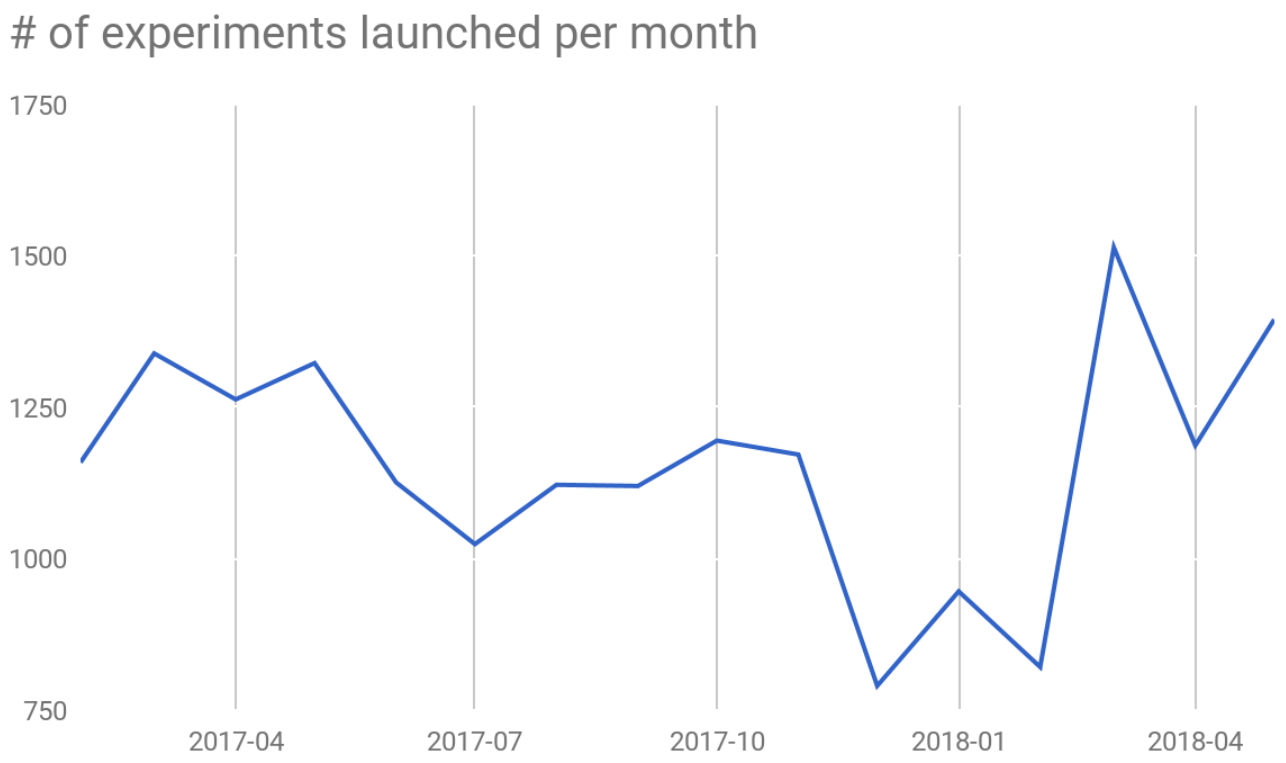


Figure 1. On average, over 1,000 experiments are running on Uber's Experimentation Platform at any given time.

Below is a chart outlining the types of experimentation methodologies that the Experimentation Platform team uses:

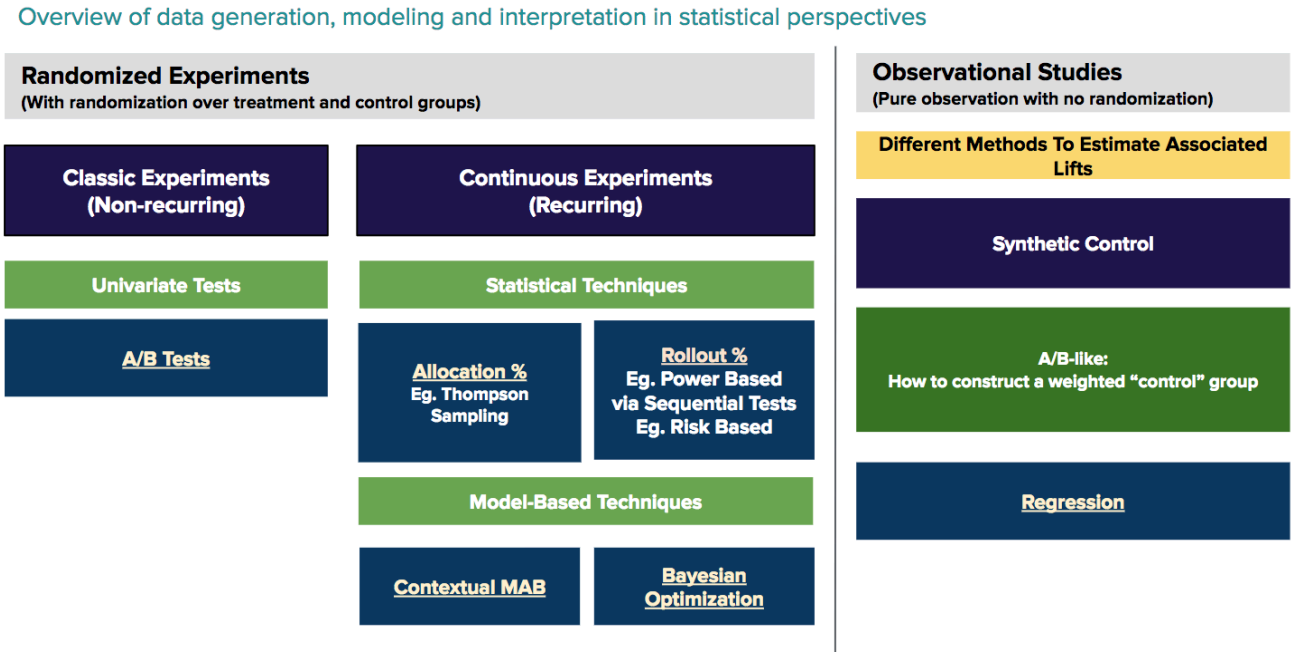


Figure 2. Uber's Experimentation Platform conducts both randomized experiments and observational studies.

There are various factors that determine which statistics methodology we should apply to a given use case. Broadly, we use four types of statistical methodologies: fixed horizon A/B/N tests (t-test, chi-squared, and rank-sum tests), sequential probability ratio tests (SPRT), causal

inference tests (synthetic control and diff-in-diff tests), and continuous A/B/N tests using bandit algorithms (Thompson sampling, upper confidence bounds, and Bayesian optimization with contextual multi-armed-bandit tests, to name a few). We also apply block bootstrap and delta methods to estimate standard errors, as well as regression-based methods to measure bias correction when calculating the probability of type I and type II errors in our statistical analyses. In this article, we discuss how each of these statistical methods are used by Uber's Experimentation Platform to improve our services.

Classic A/B testing

Randomized A/B or A/B/N tests are considered the gold standard in many quantitative scientific fields for evaluating treatment effects. Uber applies this technique to make objective, data-driven, and scientifically rigorous product and business decisions. In essence, classic A/B testing enables us to randomly split users into control and treatment groups to compare the decision metrics between these groups and determine the experiment's treatment effects.

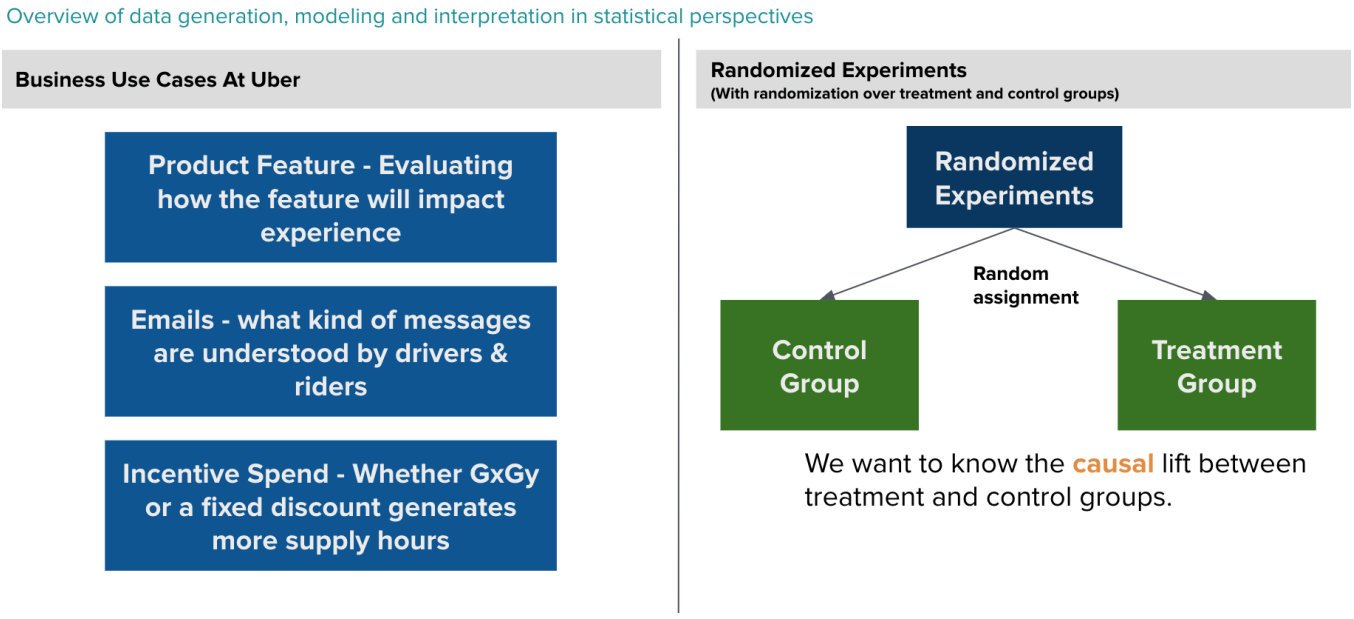


Figure 3. Uber's Experimentation Platform team conducts randomized experiments by leveraging A/B/N tests to determine the lift.

A common use case for this methodology is feature release experiments. Suppose a product manager wants to evaluate whether a new feature increases user satisfaction with Uber's platform. The product manager could use our XP to glean the following metrics: the average values of the metric in both treatment and control groups, the lift (treatment effect), whether the lift is significant, and whether the sample sizes are large enough to wield high statistical power.

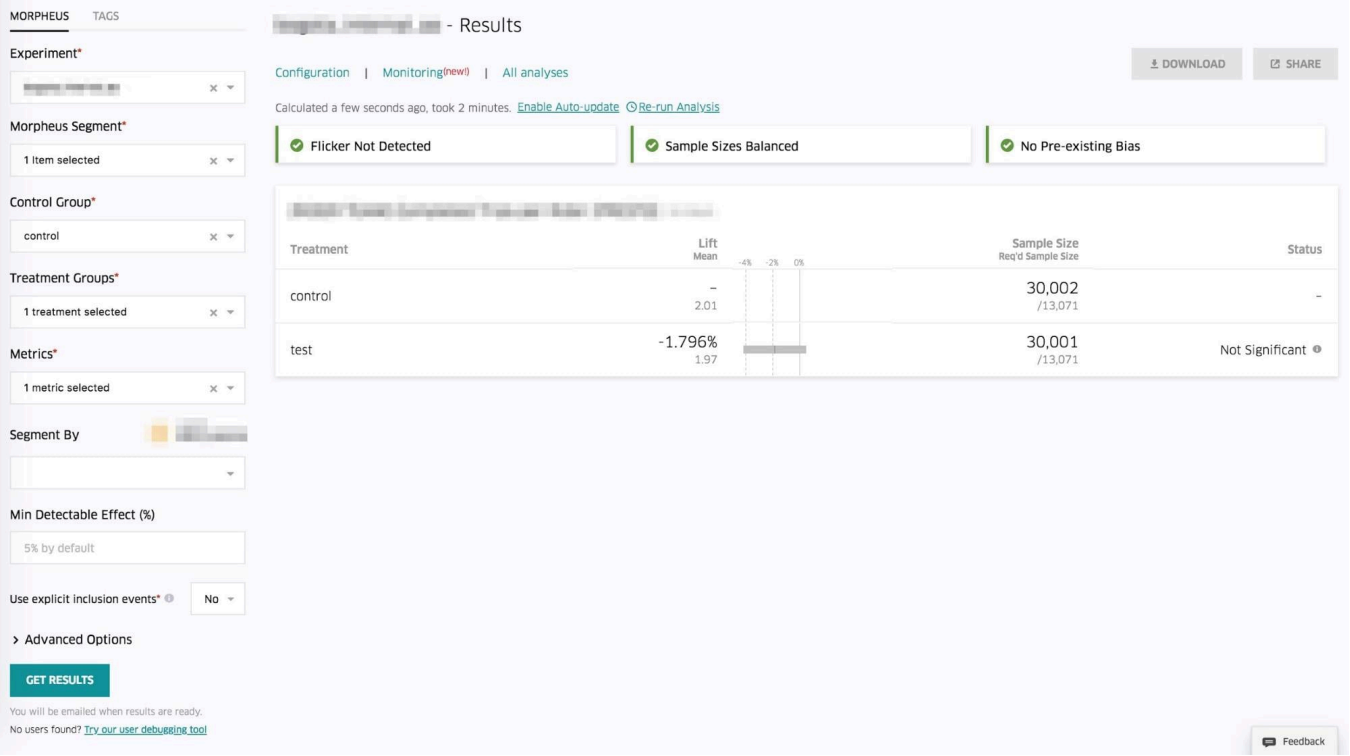


Figure 4. Our XP analytics dashboard makes it easy for data scientists and other users to access and interpret their A/B test results.

Statistics engine

One of our team's main goals is to deliver one-size-fits-most methodologies of hypothesis testing that can be applied to use cases across the company. To accomplish this, we collaborated with multiple stakeholders to build a statistics engine.

When we analyze a randomized experiment, the first step is to pick a decision metric (e.g., rider gross bookings). This choice relates directly to the hypothesis being tested. Our XP enables experimenters to easily reuse pre-defined metrics and automatically handles data gathering and data validation. Depending on the metrics type, our statistics engine applies different statistical hypothesis testing procedures and generates easy-to-read reports. At Uber, we invest heavily in the research and validation of methodologies and are constantly improving the robustness and effectiveness of our statistics engine.

Figure 5, below, offers a high-level overview of this powerful tool:

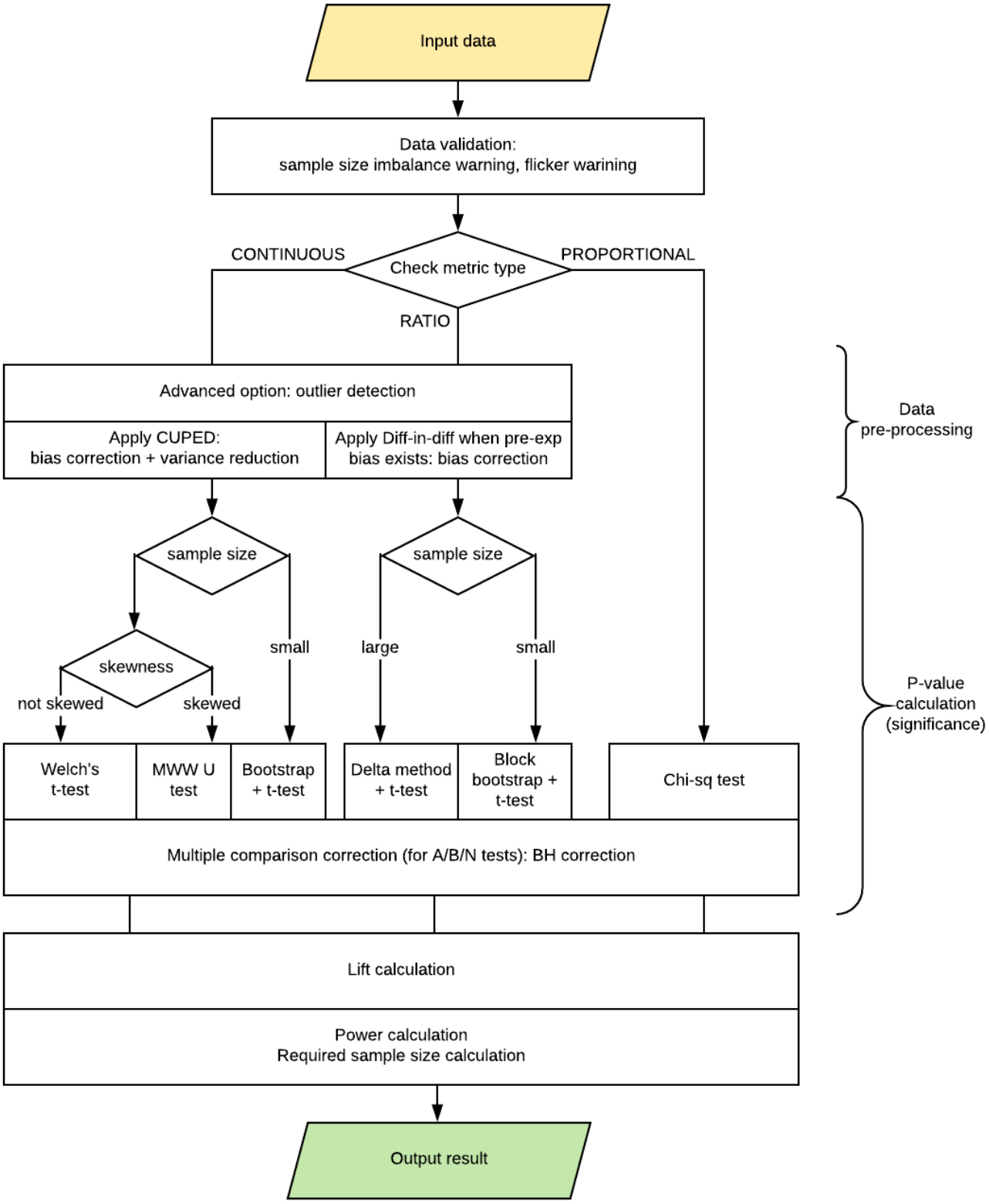


Figure 5: Uber's statistics engine is used for A/B/N experiments and dictated by fixed horizon hypothesis testing methodologies.

Key components and statistical methodologies

After gathering data, our XP's analytic platform validates the data and detects two major issues for experimenters to watch for and to keep a healthy skepticism in their A/B experiments:

- **Sample size imbalance**, meaning that the sample size ratio in the control and treatment groups is significantly different from what was expected. In these scenarios, experimenters must double check their randomization mechanisms.
- **Flickers**, which refers to users that have switched between control and treatment groups. For example, a rider purchases a new Android cell phone to replace an old iPhone, while the treatment of the experiment was only configured for iOS. The rider would switch from the treatment group to the control group. Existence of such users might contaminate the experiment results, so we would exclude these users (flickers) in our analyses.

Most of our use cases are randomized experiments and most of the time summarized data is sufficient for performing fixed horizon A/B tests. At the user level, there are three distinct types of metrics:

- **Continuous metrics** contain one numeric value column, e.g., gross bookings per user.
- **Proportion metrics** contain one binary indicator value column, e.g., to test the proportion of users who complete any trips after sign-up.
- **Ratio metrics** contain two numeric value columns, the numerator values and the denominator values, e.g., the trip completion ratio, where the numerator values are the number of completed trips, and the denominator values are the number of total trip requests.

Three variants of data preprocessing are applied to improve the robustness and effectiveness of our A/B analyses:

- **Outlier detection** removes irregularities in data and improves the robustness of analytic results. We use a clustering-based algorithm to perform outlier detection and removal.
- **Variance reduction** helps increase the statistical power of hypothesis testing, which is especially helpful when the experiment has a small user base or when we need to end the experiment prematurely without sacrificing scientific rigor. The CUPED Method leverages extra information we have and reduces the variance in decision metrics.
- **Pre-experiment bias** is a big challenge at Uber because of our diversity of users. Sometimes, constructing robust counterfactual via mere randomization just doesn't cut it. Difference in differences (diff-in-diff) is a well-accepted method in quantitative research and we use it to correct pre-experiment bias between groups so as to produce reliable treatment effects estimation.

The p-value calculation is central to our statistics engine. The p-value directly determines whether the XP reports that a result is significant. We compare the p-value to the false positive rate (Type-I error) we desire (0.05) in a common A/B test. Our XP leverages various procedures for p-value calculation, including:

- Welch's t-test, the default test used for continuous metrics, e.g., completed trips.
- The Mann-Whitney U test, a nonparametric rank sum test used to detect severe skewness in the data. It requires weaker assumptions than the t-test and performs better with skewed data.
- The Chi-squared test, used for proportion metrics, e.g., rider retention rate.

- The Delta method (Deng et al. 2011) and bootstrap methods, used for standard error estimation whenever suitable to generate robust results for experiments with ratio metrics or with small sample sizes, e.g., the ratio of trips cancelled by riders.

On top of these calculations, we use multiple comparison correction (the Benjamini-Hochberg procedure) to control the overall false discovery rate (FDR) when there are two or more treatment groups (e.g., in an A/B/C test or an A/B/N test).

The power calculation provides additional information about the level of confidence users should put into their analysis. An experiment with low power will suffer from high false negative rates (Type-II error) and high FDRs. In the power calculations our XP conducts, a t-test is always assumed. On the flipside, required sample size calculation is the opposite of a power calculation and estimates how many users are required by the experiment for it to achieve a high power (0.8).

Metrics management

As the number of the metrics used by the XP's analytics component grows (incorporating 1,000+ metrics), it becomes more and more challenging for users to determine the proper metrics to evaluate the performance of an experiment. To make it easier for new users of our analytics tool to uncover these metrics, we built a recommendation engine that facilitates the discovery of metrics available on our platform.

At Uber, there are two common collaborative filtering methods used for content recommendation: item-based and user-based methods. We primarily use an item-based recommendation engine since the characteristics of the experimenter do not typically have a strong influence on their project. For instance, if an experimenter switches to the Uber Eats team from the Rider team, it's not necessary for the algorithm to review the previous, Uber Eats-inspired choices of that experimenter when selecting metrics to evaluate.

Recommendation engine methodology

To determine how correlated two metrics are to each other, we add their popularity and absolute scores, enabling us to better understand their relationship. The two basic approaches to calculating these scores are:

- **Popularity score:** The more frequently two metrics are selected together across experiments, the higher the score assigned to their relationship. We use the Jaccard Index to help users discover the most relevant metric once they select their initial metric. This score accounts for the experimenters' metrics selection from past experiments.
- **Absolute score:** Using our XP, we can generate a pool of user samples from our metrics and calculate the Pearson correlation score of the two metrics. This accounts for serendipitous discovery; namely, the experimenter may not have considered adding a metric to the experiment since it is not directly related, but it might be moving with the user-selected metric.

After calculating these two scores, we add the score of the two steps above with relative weights on each term and recommend the metrics with the highest score to the experimenter based on their first choice of metrics.

Insights discovery

As Uber continues to scale, it becomes more and more challenging to mine our metrics knowledge base. Our recommendation engine enables both global and local teams to access the information they need quickly and easily, allowing them to improve our services accordingly.

For example, if an experimenter wants to measure the treatment effect on driver-partner supply hours, it may not be obvious to the experimenter to also add the number of trips taken by new riders as a metric, since this experiment focuses on the driver side of the trip equation. However, both metrics are important for this experiment because of the dynamics of our marketplace. Our recommendation engine helps data scientists and other users discover important metrics that may not have been obvious.

Sequential testing

While traditional A/B testing methods (for example, a t-test) inflate Type-I error by repeatedly taking subsamples, sequential testing offers a way to continuously monitor key business metrics.

One use case where a sequential test comes in handy for our team is when identifying outages caused by the experiments running on our platform. We cannot wait until a traditional A/B test collects sufficient sample sizes to determine the cause of an outage; we want to make sure experiments are not introducing key degradations of business metrics as soon as possible, in this case, during the experimentation period. Therefore, we built a monitoring system powered by a sequential testing algorithm to adjust the confidence intervals accordingly without inflating Type-I error.

Using our XP, we conduct periodic comparisons about these business metrics, such as app crash rates and trip frequency rates, between treatment and control groups for ongoing experiments. Experiments continue if there are no significant degradations, otherwise they will be given an alert or even paused. The workflow for this monitoring system is shown in Figure 6, below:

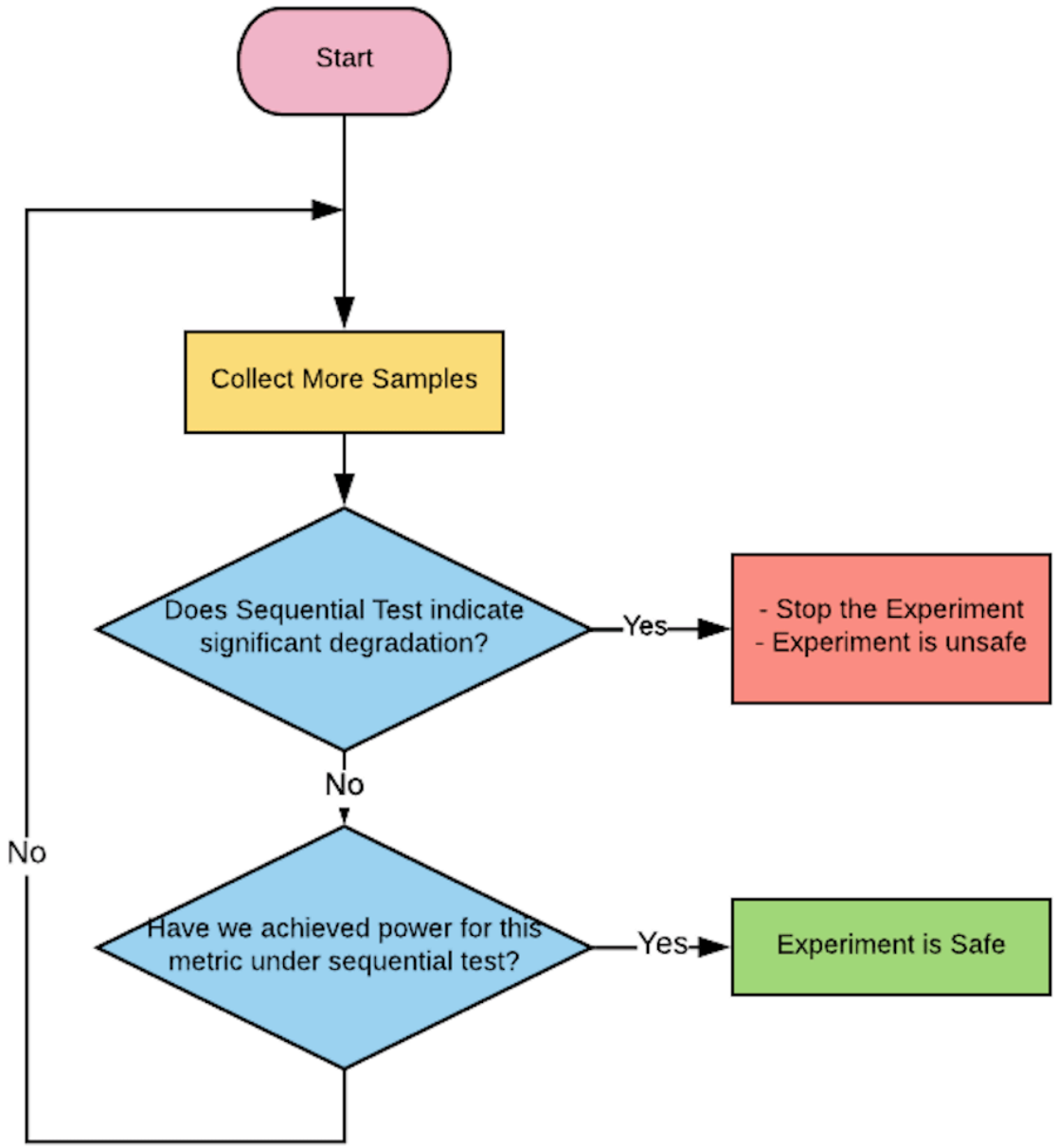


Figure 6. We integrate the sequential test methodology into the workflow of our XP's outage monitoring system.

Methodologies

We leverage two main methodologies to perform sequential testing for metrics monitoring purposes: the mixture sequential probability ratio test (mSPRT) and variance estimation with FDR.

Mixture Sequential Probability Ratio Test

The most common method we use for monitoring is mSPRT. This test builds on the likelihood ratio test by incorporating an extra specification of mixing distribution H. Suppose we are testing the metric difference with the null hypothesis being , then the test statistics could be written as = . Sinces we have large sample sizes and the central limit theorem can be applied to most cases, we use normal distribution as our mixing distribution, . This leads to easy

computation and a closed form expression for . Another useful property about this method is under null hypothesis, nH, 0 is proven to be a martingale: . Following this, we could construct confidence interval.

Variance estimation with FDR control

To apply sequential testing correctly, we need to estimate variance as accurately as possible. Since we monitor the cumulative difference between our control and treatment groups on a daily basis, observations from the same users introduce correlations which violate the assumption of the mSPRT test. For example, if we are monitoring click through rates, then the metric from one user across multiple days may be correlated. To overcome this, we use delete-a-group jackknife variance estimation/block bootstrap methods to generalize mSPRT test under correlated data.

Since our monitoring system wants to evaluate the overall health of an ongoing experiment, we monitor many business metrics at the same time, potentially leading to false alarms. In theory, either the Bonferroni or BH correction could be applied in this scenario. However, since the potential loss of missing business degradations can be substantial, we apply BH correction here and also tune in parameters (MDE, power, tolerance for practical significance, etc.) for metrics with varying levels of importance and sensitivity.

Use cases

Suppose we want to monitor a key business metric for a specific experiment, as depicted in Figure 7, below:

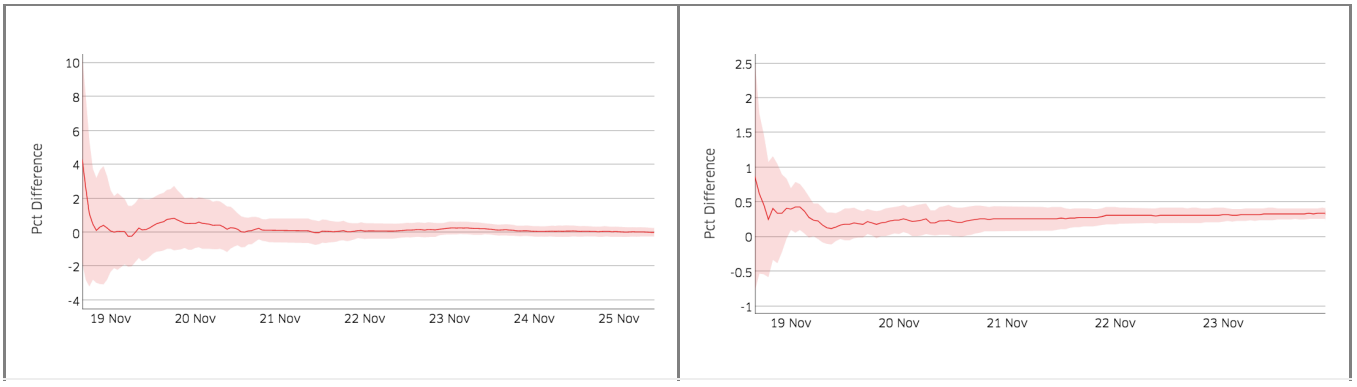


Figure 7. The sequential test methodology indicates a significant difference between our treatment and control groups, as identified in Plot B. In contrast, no significant difference is identified in Plot A.
The red lines Plots A and B signify the observed cumulative relative difference between our treatment and control groups. The red band is the confidence interval for this cumulative relative difference.

As time passes, we accumulate more samples and the confidence interval narrows. In Plot B, the confidence interval consistently deviates from zero starting on a given date, in this example, November 21. With an extra threshold (in other words, tolerance for our monitoring system) for practical significance imposed, metrics degradation is detected to be both

statistically and practically significant after a certain date. In contrast, Plot A's confidence interval shrinks but always includes 0. Thus, we didn't detect any regressions for the crash monitored in Plot A.

Continuous experiments

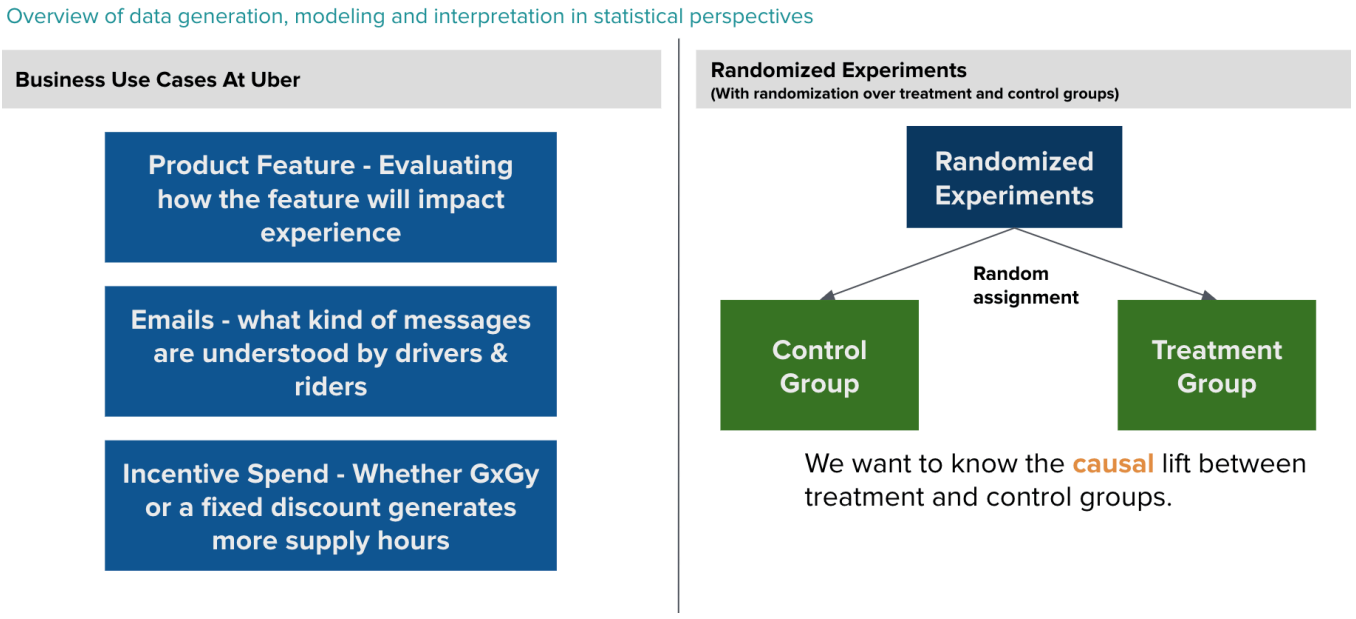


Figure 3. Uber's Experimentation Platform team conducts randomized experiments by leveraging A/B/N tests to determine the lift.

To accelerate innovation and learning, the data science team at Uber is always looking to optimize driver, rider, eater, restaurant, and delivery-partner experiences through continuous experiments. Our team has implemented bandit and optimization-focused reinforcement learning methods to learn iteratively and rapidly from the continuous evaluation of related metric performance.

Recently, we completed an experiment using bandit techniques for content optimization to improve customer engagement. The technique helped improve customer engagement compared to classic hypothesis testing methods. Figure 9, below, outlines Uber's various continuous experiment use cases, including content optimization, hyper-parameter tuning, spend optimization, and automated feature rollouts:

Case Study

Bandit / Continuous Experiments

Use Cases at Uber

1 Content Optimization
Personalization and recommendation
- In-app message for Uber Visa Card
- US Campaigns
- LATAM and EMEA Campaigns
- Mobile/Web Design Optimization

2 Hyper-Parameter Tuning
Improving the model performance
- Uber Eats Ranking
- Recommendation System for Uber Trips

3 Spend Optimization
Looking for the best strategies
- Optimal bidding strategy
- Optimal promo strategy

4 Automated Rollout
Monitoring the release
- Power and Risk based techniques
  - To decide rollout %

Figure 9. Uber's XP leverages continuous experiments for a variety of use cases, including hyper-parameter tuning and automated feature rollouts.

In Case Study 1, we outline how bandits have helped optimize email campaigns and enhance rider engagement at Uber. Here, the Uber Eats Customer Relationship Management (CRM) team in Europe, the Middle East, and Africa (EMEA) launched an email campaign to encourage order momentum early in the customer life cycle. The experimenters plan to run a campaign with ten different email subject lines and find out the best subject line in terms of the open rate and the number of open emails. Figure 10, below, details this case study:

Case Study 1: Improving Customer Engagement

Looking for best subject line in the email among 10 candidates via MAB methods

- **Topic**
  - The Marketing CRM team in EMEA wanted to test a large number of email subject lines.
- **Business Problem**
  - Uber Eats EMEA team designed new email campaign and hoped to know which subject line had the highest open rate. The team had a large number of candidates and thus it was a better use case for MAB than for AB tests to test the candidates.
- **Modeling Component**
  - Building Thompson sampling algorithm to dynamically change the allocation of treatment groups to quickly find the best treatment group.
- **Impact**
  - CRM teams from all regions as well as Finance teams started to use MAB.
  - The engineering team starts to improve the existing infrastructure to adopt continuous experiments.

A second example of how we leverage continuous experiments is parameter tuning. Unlike the first case, the second case study uses a more advanced bandit algorithm, the contextual multi-armed bandit technique, which combines statistical experiments and machine learning modeling. We use contextual MAB to choose the best parameters in a machine learning model.

As depicted in Figure 11, below, the Uber Eats Data Science team leveraged MAB testing to create a linear programming model, called the multiple-objective optimization (MOO), that ranks restaurants on the main feed of the Uber Eats app:

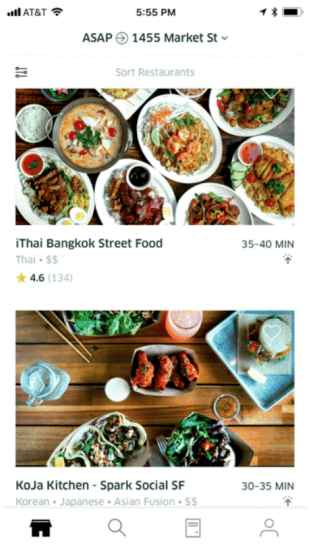
Case Study 2: Tuning Uber Eats Ranking Algorithm

Model-based CE

Multiple-objective Optimization (MOO)

Summary of Background Information

- **Topic**
  - Restaurant Recommendation of MainFeed on Uber Eats App
- **Business Problem**
  - Rank the restaurant to drive more revenue, higher conversion and higher retention rates, etc.
- **Modeling Components** [Eats DS Team]
  - **Linear programming**: an optimization problem with multiple objectives
  - **Machine learning**: prediction of business metrics
- **Statistical Methods** [Experimentation Team]
  - **A/B experiments**: testing on the null hypothesis



The algorithm behind MOO incorporates several metrics, such as session conversion rate, gross booking fee, and user retention rate. However, the mathematical solution contains a set of parameters that we need to give to the algorithm.

These experiments contain many parameter candidates for use with our ranking algorithms. The ranking results depend on the hyper-parameters we chose for the MOO model. Therefore, to improve the performance of the MOO model, we hope to figure out the best hyper-parameters via multi-armed bandits algorithm. The traditional A/B test framework is too time-intensive to handle each test, so we decided to utilize the MAB method for these experiments. MAB is able to provide a framework to quickly tune these parameters.

We chose the contextual MAB and the Bayesian optimization methods to find the maximizers of a black box function optimization problem. Figure 12, below, outlines the setup of this experiment:

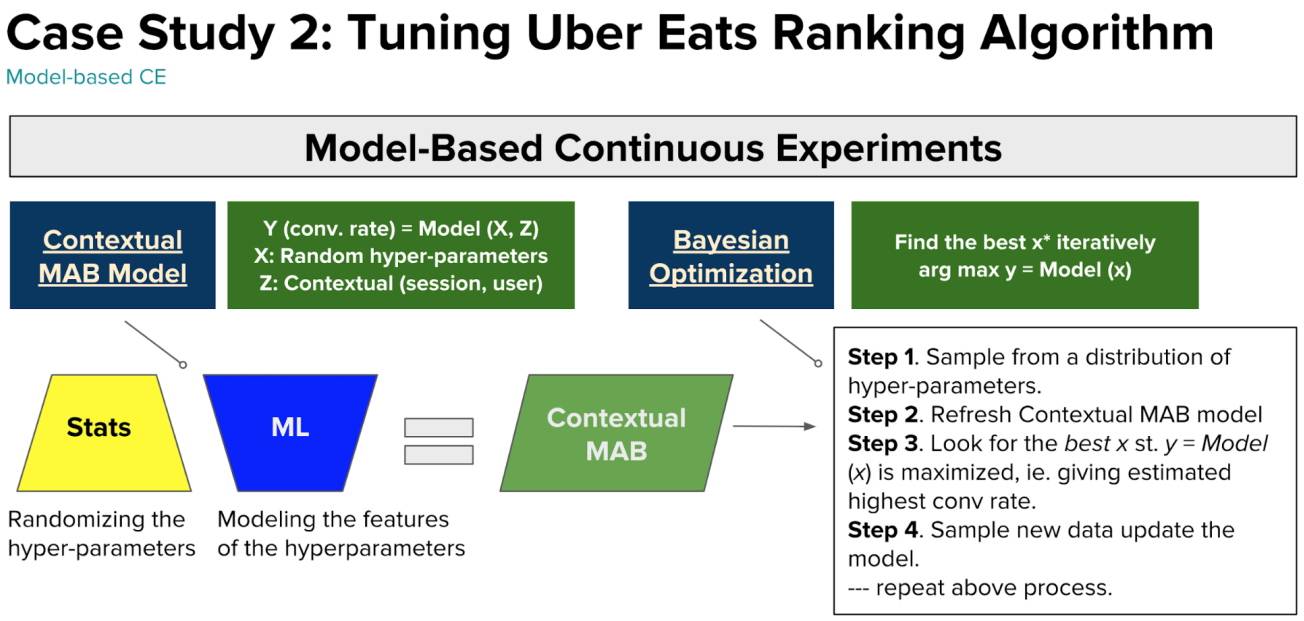


Figure 12: Our XP leverages contextual MABs for hyper-parameter tuning.

As shown above, contextual Bayesian optimization works well with both personalized information and exploration-exploitation trade-offs.

Moving Forward

As a result of its scale and global impact, Uber's problem space poses unique challenges. As our methodologies evolve, we aspire to build an ever more intelligent experimentation platform. In the future, this platform will provide insights gleaned not only from current experiments, but also previous ones, and, over time, proactively predict metrics.

Uber's Experimentation Platform team is hiring. If you are passionate about experimentation and machine learning, please apply for this role.

Subscribe to our newsletter to keep up with the latest innovations from Uber Engineering.

Anirban Deb is the former tech lead of the Experimentation, Segmentation, Personalization and Mobile App Development Platform data science teams at Uber and currently heading the Uber Freight data science organization.

Suman Bhattacharya is a senior data scientist on Uber's Experimentation Platform team.

Jeremy Gu is a data scientist on Uber's Experimentation Platform team.

Tianxia Zhou is a data scientist on Uber's Experimentation Platform team.

Eva Feng is a data scientist on Uber's Experimentation Platform team.

Mandie Liu is a data scientist on Uber's Experimentation Platform team.

Posted by Anirban Deb, Suman Bhattacharya, Jeremy Gu, Tianxia Zhou, Eva Feng, Mandie Liu

Category:

Data / ML